SPACEBOX

Новые игровые механики

Идеи, развивающие то, что в игре уже есть, и roadmap по приоритетам

31 идея в 9 областях: выживание, исследование, автоматизация, корабли, угрозы, экономика, мультиплеер, эндгейм и мировые события
На основе версии игры 0.1.5 (spec.md, ships-design.md, tutorial-design.md, Default GameSet)

Дизайн-предложение · октябрь 2026

Содержание

Об этом документе . . . 3

Обзор всех идей . . . 4

Влияние и трудоёмкость . . . 5

S Давление выживания . . . 6

X Исследование пояса . . . 8

A Автоматизация и логистика . . . 10

SH Геймплей кораблей . . . 12

T Угрозы и враги . . . 15

E Экономика и торговля . . . 16

M Мультиплеер: кооператив и PvP . . . 18

G Цели эндгейма . . . 19

EV Мировые события . . . 20

Roadmap по приоритетам . . . 22

Об этом документе

В Spacebox уже есть глубокий набор систем: воксельные астероиды с рудами и полостями, машины и электросети, трубы, кислородные комнаты, генераторы гравитации, DNA-респавн, паразиты с настоящим ИИ, корабли игроков с двигателями, пушками, консолями и столкновениями, а также мультиплеер с авторитетным сервером, claims, правилами и веб-dashboard. Не хватает **причин**: причин покинуть первый астероид, поддерживать базу, улетать далеко и играть вместе неделями.

Каждая идея ниже добавляет такую причину, **переиспользуя существующие системы**, а не изобретая параллельные. Каждая карточка называет системы, на которые опирается, и даёт заметки по движку: производительность, сеть и формат сохранений, в соответствии с правилами проекта (GameSets на данных, авторитетный сервер, необязательные ключи сохранений, никаких покадровых аллокаций, синхронизированные туториалы).

Идеи, которые пересекаются с `Spacebox/TODO.txt`, списком целей в начале `DONE.txt` или запланированным WP10 в `ships-design.md`, отмечены пометкой **пересечение**, где сказано, что нового добавляет карточка.

Как читать карточки

Трудоёмкость	S: до 1 недели · M: 1-3 недели · L: 3-6 недель · XL: более 6 недель (один разработчик вместе с агентом, включая тесты)
Приоритет	P1 ближайшие версии, большая отдача P2 среднесрочно P3 позже или по желанию
Влияние	1-5 точек: ожидаемый эффект на основной игровой цикл и удержание игроков
Опирается на	Существующие классы и системы, которые переиспользует идея (названия как в коде)

Принципы дизайна для каждой новой механики

Сначала данные	Каждая механика управляется новой JSON-секцией со значениями по умолчанию (`Fuel`, `Thruster.Fuel`, `Radiation`, `contracts.json`, `events.json`). `Default` GameSet продолжает загружаться, а моды могут настроить или отключить функцию.
Авторитетный сервер	Всё, что меняет мир или инвентари, решается на сервере и синхронизируется через `BlockStateNetwork`, `INetMessage` или `NetworkVariable`. Чисто визуальная работа остаётся на клиентах (импульс сканера, оценка комфорта).
Переключение правилом	Каждая механика давления получает переключатель `ServerRules` (через `ServerRulesService`): `decompression`, `fuel`, `mood_effects`, `world_events`, `sieges`. В одиночной игре они хранятся в `rules.json`.
Совместимые сохранения	Новые ключи NBT и JSON необязательны и имеют значения по умолчанию. `WorldUpgradeStep` нужен только для переименованных или удалённых id. Ни одной из идей не нужно менять раскладку битов `Block`.
Чистые горячие пути	Новые тикающие системы работают на уровне сущности (как `DoorSystem` / `OxygenSystem`), с частотой 1-4 Hz на сервере, ничего не аллоцируют покадрово и используют воркеры плюс `MainThreadDispatcher` для сканирования.
Туториал синхронизирован	Каждая механика, которую должен знать новичок (топливо, импульс сканера, команды), получает шаг туториала в том же изменении, а `DefaultTutorialContentTests` остаются зелёными.

Обзор всех идей

ID	Идея	Область	Объём	Приор.	Влияние
S1	Генераторы на топливе и аккумуляторные банки	Давление выживания	M	P1	●●●●●
S2	Пробоины корпуса и декомпрессия	Давление выживания	M	P2	●●●●○
S3	Радиация	Давление выживания	M	P2	●●●○○
S4	Комфорт и настроение	Давление выживания	S-M	P2	●●●○○
X1	Заброшенные корабли и структуры ПЕРЕСЕЧЕНИЕ	Исследование пояса	L	P1	●●●●●
X2	Ресурсные биомы ПЕРЕСЕЧЕНИЕ	Исследование пояса	M	P1	●●●●○
X3	Импульс сканера-разведчика	Исследование пояса	S-M	P1	●●●●○
A1	Auto-miner ПЕРЕСЕЧЕНИЕ	Автоматизация и логистика	M	P1	●●●●●
A2	Сигнальные каналы: датчики и исполнители	Автоматизация и логистика	L	P2	●●●●○
A3	Логистика: сортировщик, таблички запасов, терминал хранилищ ПЕРЕСЕЧЕНИЕ	Автоматизация и логистика	M	P2	●●●○○
A4	Грузовая катапульта (mass driver)	Автоматизация и логистика	L	P3	●●●○○
SH1	Стыковочные коннекторы и посадочные захваты ПЕРЕСЕЧЕНИЕ	Геймплей кораблей	L	P1	●●●●●
SH2	Ракетное топливо и топливные баки	Геймплей кораблей	M	P2	●●●●○
SH3	Навигационный компьютер: автопилот и круиз ПЕРЕСЕЧЕНИЕ	Геймплей кораблей	M	P1	●●●●○
SH4	Модель повреждений кораблей и разделение ПЕРЕСЕЧЕНИЕ	Геймплей кораблей	XL	P2	●●●●○
SH5	Тяговый луч и резак для разборки	Геймплей кораблей	M	P3	●●●○○
T1	Гнёзда паразитов и заражение	Угрозы и враги	M	P1	●●●●●
T2	Автоматические оборонительные турели	Угрозы и враги	M	P2	●●●●○
T3	Дроны-мусорщики и пиратские корабли	Угрозы и враги	XL	P2	●●●●●
E1	Доска контрактов (на движке туториала)	Экономика и торговля	M	P1	●●●●●
E2	Торговые аванпосты и торговцы Spacer	Экономика и торговля	L	P2	●●●●○
E3	Исследования через анализ	Экономика и торговля	M	P2	●●●●○
M1	Команды	Мультиплеер: кооператив и PvP	M	P1	●●●●○
M2	Осады claims (PvP-серверы)	Мультиплеер: кооператив и PvP	L	P3	●●●○○
M3	Таблицы лидеров и цели сервера	Мультиплеер: кооператив и PvP	S	P2	●●●○○
G1	Мегапроект Belt Gate	Цели эндгейма	L	P2	●●●●●
G2	Улей-левиафан (босс)	Цели эндгейма	L	P3	●●●●○
EV1	Режиссёр мировых событий	Мировые события	M	P2	●●●●○
EV2	Метеорные потоки	Мировые события	M	P2	●●●●○
EV3	Солнечные вспышки ПЕРЕСЕЧЕНИЕ	Мировые события	S	P2	●●●○○
EV4	Сигналы бедствия и кометы ПЕРЕСЕЧЕНИЕ	Мировые события	M	P3	●●●○○

ПЕРЕСЕЧЕНИЕ = затрагивает пункт из TODO.txt, списка целей DONE.txt или ships-design.md WP10; в карточке сказано, что она добавляет.

Влияние и трудоёмкость

Идеи в левой верхней области дают больше всего пользы на единицу работы. Большинство из них входит в фазу 1 roadmap; пункты XL справа планируются, когда готов их фундамент.



- **S** Давление выживания
- **X** Исследование пояса
- **A** Автоматизация и логистика
- **SH** Геймплей кораблей
- **T** Угрозы и враги
- **E** Экономика и торговля
- **M** Мультиплеер: кооператив и PvP
- **G** Цели эндгейма
- **EV** Мировые события

Лучшее соотношение пользы и работы

#	Идея	Влияние	Объём	Ценность
1	EV3 Солнечные вспышки	●●●○○	S	3.0
2	M3 Таблицы лидеров и цели сервера	●●●○○	S	3.0
3	X3 Импульс сканера-разведчика	●●●●○	S-M	2.7
4	A1 Auto-miner	●●●●●	M	2.5
5	E1 Доска контрактов (на движке туториала)	●●●●●	M	2.5
6	S1 Генераторы на топливе и аккумуляторные банки	●●●●●	M	2.5
7	T1 Гнёзда паразитов и заражение	●●●●●	M	2.5
8	S4 Комфорт и настроение	●●●○○	S-M	2.0
9	E3 Исследования через анализ	●●●●○	M	2.0
10	EV1 Режиссёр мировых событий	●●●●○	M	2.0

Ценность = влияние, делённое на вес трудоёмкости (S 1, S-M 1.5, M 2, L 4, XL 6). Это лишь первичная сортировка: зависимости и интерес тоже важны, см. roadmap.

S Давление выживания

Сейчас выживание сводится в основном к кислороду, заряду скафандра и паразитам. Эти идеи добавляют медленное и понятное давление, которое вознаграждает продуманную базу, а не наказывает игрока случайным образом. (4 идеи: S1, S2, S3, S4)

S1 Генераторы на топливе и аккумуляторные банки

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●●

СУТЬ ИДЕИ

Кобальтовые и ядерные энергоблоки после постройки вырабатывают энергию вечно. Дадим генераторам необязательную JSON-секцию **Fuel**: слот топлива (например, `cobalt_ingot` или `energy_cell` для кобальтовых блоков, `uranium_rods` для ядерных), время горения одного предмета и выработку только пока топливо не кончилось. Добавим тип блока **battery**, который заряжается от избытка энергии и разряжается, когда потребление выше выработки. Индикатор энергии в окнах машин показывает итоговый баланс, запас энергии и время до разрядки. Солнечные панели можно добавить, когда в мире появится направление на солнце.

ОПИРАЕТСЯ НА

`ElectricNetworkManager` (правило «выработка >= потребления» даёт `IsActive`), `GeneratorBlockData` (уже читает `Power` и `Explosive`), система контейнеров (`IContainerBlock`, снапшоты `ContainerServer`), трубы (адаптер `FuelEndpoint`, зарегистрированный через `ItemTransferEndpoints.Register`) и существующие цепочки крафта `uranium_rods` / `energy_cell`.

ОПЫТ ИГРОКА

Энергия становится постоянным циклом: добыть, переработать, загрузить в реактор. Автоматизация подачи топлива трубами ощущается заслуженной. Блэкаут важен, потому что вместе с ним встают генератор кислорода и DNA-реконструктор.

РИСКИ

Это меняет баланс «построил и забыл». Старые миры не должны внезапно лишиться энергии: топливо действует только в GameSet, где оно задано, а апгрейд заполняет каждый генератор запасом на льготный период. Батареи нарушают правило сети «всё или ничего», поэтому разряжающаяся батарея должна просто считаться выработкой.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Заряд батарей и сжигание топлива должны решаться на сервере и синхронизироваться как состояние блока (`INetworkStateBlock`, как у генератора гравитации). Сейчас клиенты сами считают электричество, и сжигание на клиенте разошлось бы с сервером. Каждая сеть тикает с частотой 1-2 Hz через `TickTaskManager` без аллокаций. Сохранения: `battery_data` { charge } и `generator_fuel` { slot, burn_left } через `IBlockSerializer`. Оба ключа необязательные, поэтому шаг апгрейда не нужен. Шаг туториала *Power* должен учить заправке (`DefaultTutorialContentTests` должны оставаться зелёными).

S2 Пробоины корпуса и декомпрессия

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Когда разрушается блок рядом с герметичной комнатой (выстрелом, взрывом, пушкой или столкновением), воздух не просто исчезает. 1-3 с он выходит через дыру: игроков, дропы и физические тела тянет к ней, слышно шипение, летит поток частиц, а двери с пометкой **bulkhead** закрываются автоматически. Добавим аварийный герметик - предмет, который ставит в дыру временный блок пены.

ОПИРАЕТСЯ НА

`OxygenSystem` / `AirFloodFill` (утечки уже обнаруживаются), `AirMasks` (разница старой и новой маски даёт потерянные клетки), код отталкивания в `ExplosionForces`, `DoorSystem` (автозакрытие), `Oxygen.OnBlockChanged`, `AirZoneSounds`, частицы.

ОПЫТ ИГРОКА

Драматичные, физичные моменты. Шлюзы и отсеки начинают окупаться, а у боя кораблей появляются последствия помимо выбитых блоков.

РИСКИ

Flood fill запускается при каждой пробоине, но буферы берутся из пула, а радиус ограничен 32. Когда тебя раз за разом высасывает наружу, это раздражает, поэтому сила ограничена, действует только в режиме выживания и отключается серверным правилом `decompression`.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Пробоина = прошлая маска воздуха минус новая маска той же сущности: число потерянных клеток и их центр, плюс удалённая клетка как дыра. Сервер шлёт `DecompressionMessage` (сущность, клетка дыры, сила). Каждый клиент толкает только своего игрока (модель взрывов), а дропы и тела двигает сервер. Ничего не сохраняется. JSON: `Door.AutoCloseOnLeak` (по умолчанию false).

S3 Радиация

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●○○

СУТЬ ИДЕИ

Урановая руда, урановые стержни и ядерные энергоблоки излучают радиацию. В режиме выживания накапливается **доза**, которая снижает максимальное здоровье, пока её не снимет антирад или время. Плотные блоки (тяжёлые металлы, титан) экранируют. Треск счётчика Гейгера следует за мощностью дозы, а снаряжение-сканер показывает уровень.

ОПИРАЕТСЯ НА

Паттерн `GravityField.Sample` (источники регистрируются через `IEntityBlockListener` и суммируются) превращается в `RadiationField`. Также `StatsData` / здоровье, `EquipmentStatsBinder` (защита брони, как `ExplosionProtection`), `Mass` блока как экранирование, `SuffocationDamage` как модель урона во времени, расходники.

ОПЫТ ИГРОКА

У ядерной энергии и добычи урана появляется цена, а у тяжёлой брони и экранированных реакторных залов - смысл. Звук Гейгера добавляет напряжения.

РИСКИ

Невидимый урон кажется нечестным, поэтому нужны сильная звуковая и визуальная обратная связь и медленный набор дозы. Урановая руда должна оставаться слабой, чтобы обычная добыча была безопасной.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Замер 4 раза в секунду для локального игрока, только по N ближайшим источникам. Экранирование считается DDA-лучом `VoxelLine`, суммирующим массу блоков, с кэшем на пару источник/игрок, пока не изменится `DataVersion` чанка. Считается на клиенте для собственного тела игрока (как урон от падения). Сохранения: `Radiation` в JSON игрока (по умолчанию 0) и JSON `Radiation { Strength, Range }` у блоков (по умолчанию нет).

S4 Комфорт и настроение

Трудоёмкость S-M **Приоритет** P2 **Влияние** ●●●○○

СУТЬ ИДЕИ

`Mood` уже существует (компонент, полоска в HUD, события по таймеру и звуки настроения), но почти не влияет на геймплей. Превратим его в **комфорт**: герметичная комната получает оценку за свет, окна, картины и таблички, гравитацию, блоки сидений или кроватей. Высокое настроение ускоряет регенерацию здоровья и заряда скафандра. Низкое (долгое пребывание в темноте, без воздуха или рядом с паразитами) немного замедляет бурение. 30 картин, которые можно скрафтить, наконец обретают смысл.

ОПИРАЕТСЯ НА

`Mood` / `TimedMoodEvent`, `SealedRoomFill` / `GravityRoom` (определение комнат), `World.IsBreathable`, рамки картин и таблички, `LightManager`, `PlayerStatistics`.

ОПЫТ ИГРОКА

Вознаграждает строительство дома, а не коробки. Механика мягкая и никогда не наказывает.

РИСКИ

Игроки могут спамить декором, поэтому каждый тип декора даёт убывающую отдачу. Механика должна оставаться необязательной (правило `mood_effects`).

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Комната оценивается только когда игрок в неё входит или меняются её чанки (`DataVersion`), на воркере по списку клеток комнаты. Покадрового flood fill нет. Оценка локальна для каждого игрока, поэтому по сети ничего не передаётся. Сохранения: `Mood` в JSON игрока (по умолчанию 50).

X Исследование пояса

В секторе 1250-2500 астероидов, но большинство из них выглядят и играются одинаково, а файлы структур пусты. Эти идеи дают игроку причины покинуть первый астероид. (3 идеи: X1, X2, X3)

X1 Заброшенные корабли и структуры

Трудоёмкость L **Приоритет** P1 **Влияние** ●●●●●

ПЕРЕСЕЧЕНИЕ TODO.txt "generation"; заголовок DONE.txt "better generation (different objects and resources)". Добавляет конкретный дизайн для пустых файлов структур.

СУТЬ ИДЕИ

Наполнить пустые определения `abandoned_ship`, `alien_base` и `ruins`. Структура - это prefab-файл `.entity`, собранный в креативе, плюс JSON: веса биомов, количество на сектор, таблицы лута для каждого хранилища, проход повреждений (блоки случайно удалены или треснуты), необязательные мобы (гнездо паразитов) и необязательные тексты табличек (логи и лор). В заброшенных кораблях стоят настоящие корабельные блоки, так что игрок может починить их ремонтным инструментом, подать энергию и улететь.

ОПИРАЕТСЯ НА

Файлы сущностей не зависят от позиции (любой `.entity` можно скопировать в любой сектор), заглушки `NotGeneratedEntity`, `Generation/Loot` (`chest`, `alien-chest` уже есть), список `Structures` в `generator.json` (`GenerationLoader.LoadStructures` сейчас закомментирован), ремонтный инструмент и стадии трещин, таблички, `ShipBody` (любая сущность с корабельными блоками становится кораблём), `EntityProtection` для сюжетных мест.

ОПЫТ ИГРОКА

Это самый большой недостающий элемент исследования: что-то, что можно найти между астероидами. Заброшенный корабль может стать ранним кораблём ещё до полной технологии, а таблички и картины рассказывают историю через окружение.

РИСКИ

Prefab'ы требуют времени на создание. Важен баланс лута (никакого дешёвого титана). Полное сохранение каждой структуры раздуло бы миры, поэтому структуры должны регенерироваться, как астероиды.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

`StructureEntity` (или флаг на `SpaceEntity`) с `SavesOnlyChangedChunks` и NBT-ключами `structure_id` + seed (со значениями по умолчанию). Чанки prefab'а декодируются один раз и кэшируются. Проход повреждений детерминирован от seed, поэтому клиенты сами строят нетронутые структуры и по сети ничего не идёт, как с астероидами. Лут разыгрывается на сервере при первом открытии (как у Spacer). Тест: размещение prefab'а детерминировано и не выходит за лимит в 16 чанков.

X2 Ресурсные биомы

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●○

ПЕРЕСЕЧЕНИЕ Заголовок DONE.txt "better generation (different objects and resources)".

СУТЬ ИДЕИ

Сейчас биомов два (`asteroid_field`, `dense_cluster`). Сделаем биомы причиной путешествовать: тулий, обскурий и уран будут в основном в определённых биомах дальше от центра сектора: **Ice Ring** (лёд и метан), **Iron Graveyard** (тяжёлые металлы, много заброшенных кораблей), **Obscurium Rift** (тёмный, густой туман, редкие астероиды, больше паразитов), **Radiant Cluster** (уран, радиация). Каждый биом задаёт цвет и плотность тумана, оттенок неба, фоновый звук и правила спавна мобов.

ОПИРАЕТСЯ НА

`BiomeGenerator` / `BiomesMap`, `MinDistanceFromCenter` / `MaxDistanceFromCenter` (уже есть в JSON биомов), пресеты астероидов и `AsteroidOreGenerator`, `fog.glsl` с плотностью тумана в `GlobalUniforms`, правила `Spawn` мобов и превью карты биомов в редакторе генерации.

ОПЫТ ИГРОКА

Игроки строят в голове карту пояса. У кораблей и топлива появляются направления, а радар и waypoints становятся важны.

РИСКИ

Нетронутые астероиды в существующих мирах сгенерируются иначе, если карта биомов изменится. Нужно сохранить существующие id биомов и добавлять биомы только в новых поясах расстояний, либо ограничить их версией через `Version` в `generator.json`.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Биом в точке камеры определяется раз в секунду, туман и фоновый звук плавно смешиваются (без покадровых аллокаций). Всё задаётся в JSON со значениями по умолчанию. `Default` GameSet ломать нельзя: новые биомы входят в Default, но только за пределами текущего пояса спавна.

X3 Импульс сканера-разведчика

Трудоёмкость S-M **Приоритет** P1 **Влияние** ●●●●○

СУТЬ ИДЕИ

Снаряжение-сканер (базовое, железное, титановое) получает активный импульс. Игрок удерживает клавишу, через 1 с расходится сонарная волна и на несколько секунд подсвечивает сквозь породу скопления руды в радиусе, раскрашенные по типу руды. Импульс также оставляет на астероиде временную метку разведки с оценкой содержимого. Блок-анализатор остаётся инструментом для точных цифр.

ОПИРАЕТСЯ НА

Настройки радара (`Range`, `ShowMass`), `AnalyzerScanner` (оценки массы и минералов), `TagManager` / waypoints, детерминированный `AsteroidOreGenerator`, линии в мире из `ShipOutlineRenderer` / `ClaimVisuals`.

ОПЫТ ИГРОКА

Добыча становится прицельной вместо копания вслепую. Это приятное действие со звуком и заодно идеальный шаг туториала.

РИСКИ
Рентгеновское зрение может сделать исследование тривиальным, поэтому у импульса есть перезарядка и стоимость в энергии, он показывает скопления, а не отдельные блоки, и его радиус зависит от тира.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)
Сканирование на воркере по палитрам загруженных чанков (чанки без руды в палитре пропускаются, поэтому это дёшево), результаты сливаются в боксы скоплений и возвращаются через `MainThreadDispatcher`. Рисуется не более 64 боксов одним батчем линий. Работает только на клиенте, без сохранений и сетевого трафика. Нужны input action с клавишей по умолчанию и шаг туториала.

A Автоматизация и логистика

Трубы, машины и электросети уже есть. Не хватает производства, которое работает без участия игрока, и управления этим производством. (4 идеи: A1, A2, A3, A4)

A1 Auto-miner

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●●

ПЕРЕСЕЧЕНИЕ Цель автора "automation" (заголовок DONE.txt).

СУТЬ ИДЕИ
Многоклеточный блок **miner**, который ставится вплотную к породе астероида. Он бурит блоки перед собой туннелем или слоями со скоростью и тиром из JSON (как предметы-буры), потребляет энергию и складывает дроп во внутренний буфер, который опустошают трубы. Улучшается существующими алюминиевыми, железными и титановыми буровыми насадками.

ОПИРАЕТСЯ НА
Тиры буров и `PowerToDrill`, `BlockDestroyer.Destroy` (дроп, статистика, сеть), `IItemTransferEndpoint` и трубы, `ElectricNetworkManager`, `IReservedAreaBlockData` для буровой головки, `DropManager`. Паразиты слышат шум через `HearingRange`.

ОПЫТ ИГРОКА
Это цель *автоматизации*: база, которая производит, пока игрок исследует, и связывает трубы, печь и хранилища в настоящие фабрики.

РИСКИ
В мультиплеере это может стать инструментом гриферов, поэтому он соблюдает `ProtectionRules` и claims. Перестройка мешей стоит ресурсов, поэтому каждый miner ограничен одним блоком в секунду и использует бюджет re-mesh чанков.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)
Серверный `TickTask` (как `DisassemblerBlock`). Целевые клетки ищутся в пространстве сущности задетого астероида (`Sector.Raycast`, если miner стоит на другой сущности). Удаления рассылаются как обычные правки блоков через `WorldEditNetwork`, и нужно выставлять `chunk.IsModified`. Сохранения: `miner_data` (прогресс, курсор паттерна, слоты буфера) со значениями по умолчанию.

A2 Сигнальные каналы: датчики и исполнители

Трудоёмкость L **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Лёгкий логический слой на каждую сущность, который использует цветовые группы, уже знакомые игрокам по корабельным консолям. **Датчики:** кнопка, рычаг, нажимная плита присутствия, уровень хранилища, уровень энергии, наличие воздуха, таймер. **Исполнители:** двери, генераторы гравитации, трубы (вкл/выкл), свет, генератор кислорода, а также двигатели и пушки (уже управляются консолью). Каждому блоку назначается цвет канала инструментом привязки. Канал включён, если включён любой из его датчиков (OR), а флаг инверсии даёт NOT.

ОПИРАЕТСЯ НА

`ConsoleBindings` / `ShipControlSystem.ApplyCommand` (исполнители уже являются `IShipPart`), `ShipBindingTool` и `ShipOutlineRenderer` (цветные контуры), `DoorNetwork`, `GravityConfigMessage`, отключение по питанию через `IsActive`, `BlockStateNetwork`.

ОПЫТ ИГРОКА

Самоциклирующиеся шлюзы, свет, включающийся при входе, автоматическое отключение при нехватке энергии, управление фабрикой. Песочница для инженеров без языка программирования.

РИСКИ

Расползание сложности (это не должно стать Redstone). Петли исключены: каналы вычисляются один раз за тик без цепочек внутри тика.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

`SignalSystem` на каждую сущность (как `DoorSystem`) с 16-битной маской каналов. Назначения каналов хранятся в карте сериализатора (`signal_data`: клетка, канал, инверсия), а не в битах `Block`, так что горячая структура не меняется. Сервер вычисляет каналы и отправляет один `ushort` на сущность только при изменении. В старых сохранениях данных нет, то есть нет и каналов.

A3 Логистика: сортировщик, таблички запасов, терминал хранилищ

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●○○

ПЕРЕСЕЧЕНИЕ Связано с TODO.txt "inventory refactoring".

СУТЬ ИДЕИ

Sorter pipe (несколько слотов фильтра, направляет предметы по сторонам), выходы для переполнения и приоритета, токен табличек `<count=item>`, который показывает запас в присоединённом ящике, и **storage terminal**, который перечисляет все ящики на сущности и забирает предметы из любого из них.

ОПИРАЕТСЯ НА

`PipeBlock` (слот фильтра, стороны входа/выхода, round robin), `StorageEndpoint`, токены `SignMarkup` и `SignTextBaker`, `ContainerServer`, окно крафта (оно могло бы брать ингредиенты из терминала).

ОПЫТ ИГРОКА

Меньше жонглирования инвентарём, а базы выглядят живыми, потому что таблички показывают запасы.

РИСКИ

Терминал на сотни ящиков дорог, поэтому итоги кэшируются на сущность и обновляются только при изменении хранилищ.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Сортировщик - вариант `PipeBlockData` со списком фильтров (расширение `pipe_data`). Таблички запасов вычисляются на сервере и передаются как состояние блока, так что клиенты никогда не сканируют хранилища. Только необязательные ключи.

A4 Грузовая катапульта (mass driver)

Трудоёмкость L **Приоритет** P3 **Влияние** ●●●○○

СУТЬ ИДЕИ

Блок-катапульта выстреливает герметичную грузовую капсулу в сторону парного маяка-приёмника на другой сущности. Приёмник ловит капсулы в радиусе и выгружает их в трубы. Капсулы можно сбить, потерять или украсть.

ОПИРАЕТСЯ НА

`SphereBody` / `PhysicsWorld`, модель репликации дропов (`DropManager.Replicator`), метки, отталкивание взрывом, трубы.

ОПЫТ ИГРОКА

Логистика масштаба пояса и спонтанное пиратство в мультиплеере.

РИСКИ

Капсулы могут теряться в выгруженных секторах, а обычные физические тела пока не синхронизируются по сети.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Капсулы - сетевые объекты, принадлежащие серверу, как дропы (сообщения спавна и удаления, локальная симуляция). В открытом космосе нет гравитации, поэтому траектория прямая и момент прибытия можно вычислить на сервере, даже если сектор выгружен. Сохраняются в данных сектора (`cargo_pods`, необязательно).

SH Геймплей кораблей

Корабли летают, стреляют и сталкиваются (WP1-WP9 из ships-design.md готовы). Эти идеи превращают корабли из отдельной функции в игровой цикл: топливо, перелёты, стыковка и повреждения. (5 идей: SH1, SH2, SH3, SH4, SH5)

SH1 Стыковочные коннекторы и посадочные захваты

Трудоёмкость L **Приоритет** P1 **Влияние** ●●●●●

ПЕРЕСЕЧЕНИЕ Запланировано в ships-design.md WP10. Добавляет связывание сетей и производительность сервера.

СУТЬ ИДЕИ

Connector соединяется с другим коннектором лицом к лицу. Корабли сцепляются и пилотируются как один, а энергия, проход и трубы соединяются через стык. **Landing clamp** фиксирует корабль на астероиде и делает его статичным: никаких затрат на физику, и сдвинуть его нельзя.

ОПИРАЕТСЯ НА

`ShipBody`, `SpaceEntity.SetPose`, `ElectricNetworkManager` на каждую сущность, `ItemTransferEndpoints`, `ProtectionRules.CanUseEntity` и обработка пар в `ShipCollider`.

ОПЫТ ИГРОКА

Корабли-носители и шаттлы, дозаправка на домашней базе, припаркованные флоты.

РИСКИ

Составное твёрдое тело - это сложно (общая инерция, столкновения внутри пары). Первая версия должна кинематически прикреплять пристыкованный корабль к позе родителя, чтобы симулировался только родитель.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Обе сущности сохраняют связь стыковки (id `dock_parent` плюс относительная поза, по умолчанию нет). `ShipStateMessage` передаёт только родителя, а клиенты вычисляют позу дочернего корабля. Электрический мост: виртуальное ребро между двумя сетями, чтобы их суммы считались вместе. Зафиксированные корабли полностью пропускают `ShipCollider` - реальная экономия CPU на больших серверах.

SH2 Ракетное топливо и топливные баки

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Химические двигатели сжигают топливо из блоков-баков: `liquid_hydrogen` и `compressed_methane`, которые уже можно скрафтить, но почти не используются. Ионному двигателю нужна только энергия, но он слабый. Баки - это контейнеры, которые заполняются трубами, как любое хранилище. HUD консоли показывает остаток топлива и примерную дальность.

ОПИРАЕТСЯ НА

`ThrusterBlock` (`Force`, `Power`), `AutoFlight`, `IContainerBlock`, трубы, цепочки «лёд - вода - водород» в `recipes.json`, ледяные биомы (X2).

ОПЫТ ИГРОКА

Перелёты требуют планирования, ледяные астероиды становятся стратегическими, у очистительных заводов появляется смысл, а в кооперативе - цикл спасения застрявших кораблей.

РИСКИ

Застрять обидно, поэтому запасной вариант - ионный двигатель, топливо можно отключить серверным правилом, а в креативе оно не расходуется.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Расход считается на сервере каждый тик для каждой работающей группы двигателей и суммируется по сущности. Уровень заполнения бака - состояние блока для HUD, а само топливо лежит в слотах контейнера (существующая сериализация). Необязательный JSON `Thruster.Fuel { Item, PerSecond }`, по умолчанию нет, так что существующий контент не меняется. Шагам туториала *Thrusters* и *Fly!* нужна задача на топливо, если Default его использует.

SH3 Навигационный компьютер: автопилот и круиз

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●○

ПЕРЕСЕЧЕНИЕ Автопилот запланирован в ships-design.md WP10. Круизный режим - новый.

СУТЬ ИДЕИ

Навигационный блок позволяет пилоту выбрать waypoint или метку и включить автопилот: `AutoFlight` разворачивает корабль и разгоняется, тормозит перед прибытием и обходит астероиды с помощью лучей-зондов. **Круиз:** когда в пределах N блоков нет сущностей, `MaxSpeed` растёт (например, с 20 до 120) с эффектом скоростных полос и снова падает рядом с препятствиями.

ОПИРАЕТСЯ НА

`AutoFlight` (PD-регулятор), `ShipPhysicsSettings.MaxSpeed`, `WaypointService` / `TagManager`, `Sector.Raycast`, `EntitySectorTracker`, `WorldObservers` (стриминг на сервере).

ОПЫТ ИГРОКА

Сектор шириной 8192 блока. При 20 блоках/с его пересечение занимает около 7 минут, поэтому именно круиз делает дальние биомы (X2) достижимыми. ИИ пиратов (T3) нужен тот же автопилот.

РИСКИ

Стриминг может не успевать на высокой скорости, поэтому круизная скорость ограничивается обратной связью от `StreamingBudget` и очереди генерации сервера. Также нужно следить за точностью float (`RenderSpace`).

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Автопилот работает там, где симулируется корабль (на сервере), а цель сохраняется в состоянии консоли (`nav_data`, необязательно). Сетевой протокол не меняется (`ShipStateMessage`). Заглушки секторов подгружаются заранее вдоль вектора скорости.

SH4 Модель повреждений кораблей и разделение

Трудоёмкость XL **Приоритет** P2 **Влияние** ●●●●○

ПЕРЕСЕЧЕНИЕ Запланировано в ships-design.md WP10. Добавляет HUD критических систем и интеграцию с ремонтом.

СУТЬ ИДЕИ

Столкновения повреждают блоки пропорционально энергии удара. Пушки могут выводить из строя подсистемы: обесточенная консоль или сломанные двигатели, которые уводят корабль в сторону. Когда корабль разрезан надвое, отделившаяся часть становится отдельной сущностью и сохраняет скорость. Консоль показывает вид критических систем с подсветкой повреждённых частей.

ОПИРАЕТСЯ НА

Импульсы `ShipContactSolver`, `Chunk.DamageBlock` / `BlockDamage.Apply` и визуал трещин, проверка связности на пуловых буферах в стиле `AirFloodFill`, `EntityCreationNetwork`, `ShipOutlineRenderer`, ремонтный инструмент.

ОПЫТ ИГРОКА

Бой кораблей обретает вес, становится возможен таран, а обломки можно разбирать.

РИСКИ

Разделение больших сущностей дорогое и вызывает всплески сетевого трафика. У астероидов бесконечная масса, поэтому удар о скалу может слишком легко разбивать корабли - нужны порог урона и серверное правило.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

После удалений на динамических сущностях проверка связности ставится в очередь на воркер (debounce 0,5 с). Сервер создаёт новую сущность, переносит клетки напрямую (`ClearCellSilently` / `PlaceCellRaw`, как делают двери) и рассылает `EntityCreatedMessage` плюс пакетную правку. Каждая часть сохраняется как отдельный файл сущности, так что формат сохранений не меняется.

SH5 Тяговый луч и резак для разборки

Трудоёмкость M **Приоритет** P3 **Влияние** ●●●○○

СУТЬ ИДЕИ

Тяговый блок или ручной инструмент притягивает дропы, капсулы и небольшие сущности ниже лимита массы и может буксировать заброшенные корабли. Резак разбирает блоки ничейных заброшенных кораблей на компоненты быстрее, чем добыча.

ОПИРАЕТСЯ НА

Код отталкивания `SphereBody`, силы `ShipBody`, приложенные в точках, рецепты разборки, схема взаимодействия ремонтного инструмента.

ОПЫТ ИГРОКА

Более аккуратная разборка обломков и сбор лута после боёв.

РИСКИ

Буксировка кораблей с игроками на борту; механика должна соблюдать claims.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Сила прикладывается на сервере к `ShipBody` цели (новой синхронизации не нужно). Резак использует `BlockDestroyer` с выходом рецептов разборки.

T Угрозы и враги

Минеральный паразит - единственный враг. Эти угрозы опираются на уже существующие системы мобов, пушек и ИИ. (3 идеи: T1, T2, T3)

T1 Гнёзда паразитов и заражение

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●●

СУТЬ ИДЕИ

Внутри некоторых астероидов генерируются **ульи** паразитов (органические блоки), чаще в биоме Rift. Улей - источник спавна: паразиты появляются рядом с ним, а не в любом неосвещённом месте. Он медленно превращает окружающую породу в заражённую (из неё выпадает биомасса), а его уничтожение прекращает спавн на этом астероиде. Большие ульи охраняет вариант-королева.

ОПИРАЕТСЯ НА

Правила `MobSpawner` (неосвещённый воздух рядом с природными блоками), `MobSenses`, состояния `IMobBrain` (поиск пищи, поедание), таблицы лута, `LightManager`, мобы только на данных (новый вариант - просто JSON), звуки паразитов.

ОПЫТ ИГРОКА

Понятные цели (зачистить гнездо), стратегия безопасности базы (осветить и загерметизировать) и угроза, которая растёт по мере удаления от дома.

РИСКИ

Распространение заражения могло бы вызывать много перестроек мешей, поэтому оно идёт не быстрее 1 блока в минуту на улей и только рядом с игроками (то же правило, что и для трапез паразитов).

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Улей - тип блока, который регистрируется в `HiveRegistry` через `IEntityBlockListener`, а спавнер сначала проверяет ульи в радиусе. Заражённые блоки - новые id блоков только на данных. Распространение идёт обычными правками блоков на сервере. Позиции ульев детерминированно берутся из `AsteroidVoxelDataGenerator`, так что нетронутые астероиды ничего не сохраняют.

T2 Автоматические оборонительные турели

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Блок турели с собственным наведением: мобы, враждебные дроны и метеоры, а при включённом PvP - игроки других команд. Каждый выстрел стоит энергии, а боеприпасы берутся из присоединённого хранилища (лазерным турелям нужна только энергия).

ОПИРАЕТСЯ НА

`CannonBlock` (конус наведения, `TurnSpeed`, `CannonTurretRenderer`, `CannonLineOfFire`), наведение в стиле `MobSenses`, авторитетные снаряды `CombatNetwork`, серверное правило `Pvp`, участники claim как свои.

ОПЫТ ИГРОКА

Оборона базы против T1, T3 и EV2.

РИСКИ

Турели не должны неожиданно убивать игроков: свой или чужой определяется по участникам claim и командам (M1), а по игрокам турель стреляет только при явной настройке.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Цели выбираются на сервере с частотой 4 Hz из тел `World.Physics` и списка `MobSystem`, с проверкой прямой видимости через `Sector.Raycast`. Наведение и выстрелы используют синхронизацию пушек. Сохранения: `turret_data` (режим, фильтр целей).

T3 Дроны-мусорщики и пиратские корабли

Трудоёмкость XL **Приоритет** P2 **Влияние** ●●●●●

СУТЬ ИДЕИ

Корабли под управлением ИИ, собранные из prefab-файлов `.entity` (как заброшенные корабли) из настоящих корабельных блоков. ИИ-пилот ведёт их через `AutoFlight` и наводит их пушки. Поведения: **дрон-мусорщик** крадёт дроп и убегает, **рейдер** атакует ценные постройки игроков и в первую очередь бьёт по двигателям, а **патруль** охраняет заброшенные корабли. Уничтоженные корабли ИИ становятся материалом для разборки.

ОПИРАЕТСЯ НА

`ShipControlSystem` и `ConsoleCommand` (ИИ пишет те же команды, что и пилот), `AutoFlight`, `ShipWeapons`, `StateMachine<TContext>` и `PathScheduler` из `Engine/AI`, паттерны `MobSpawner` (только сервер, деспавн по расстоянию), NPC Spacer как экипаж для лута.

ОПЫТ ИГРОКА

У пушек появляется настоящая цель, а выход за пределы базы становится напряжённым.

РИСКИ

Большой объём работы. ИИ, врезающийся в астероиды, выглядит глупо. Много кораблей ИИ нагружают CPU серверов, поэтому есть лимит на сектор, а бездействующие деспавнятся.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

`ShipAiBrain` работает только на сервере и подаёт `ConsoleCommand`. Корабли спавнятся из prefab'ов через `EntityCreationNetwork` и не сохраняются, если не повреждены и не захвачены (флаг `ai_ship`). Синхронизация идёт через существующий `ShipStateMessage`. Зависит от SH3 (автопилот) и X1 (prefab'ы).

E Экономика и торговля

Кроме крафта, экономики нет. Эти идеи используют уже существующие системы (задачи туториала, NPC Spacer, анализатор), чтобы добавить цели и обмен. (3 идеи: E1, E2, E3)

E1 Доска контрактов (на движке туториала)

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●●

СУТЬ ИДЕИ

Блок-терминал контрактов (а позже торговые аванпосты) предлагает три сменяющихся контракта, собранных из шаблонов в GameSet `contracts.json`: доставить предметы, добыть N единиц руды, просканировать астероид, зачистить гнездо паразитов, долететь до waypoint, построить и запитать машину. Награды - предметы, открытие чертежей (E3) или репутация.

ОПИРАЕТСЯ НА

Почти вся среда выполнения туториала: типы задач (`have_item`, `destroy_block`, `process`, `craft`, `ship_travel`, `stat`, ...), `TutorialTaskFactory`, события `PlayerActivity`, `ITutorialContext`, панель `TutorialUI` и всплывающие уведомления, а также правило, что наградой никогда не бывает предмет, который просит задача.

ОПЫТ ИГРОКА

Бесконечные короткие цели после окончания туториала и мягкое направление к новым биомам и системам.

РИСКИ

`TutorialRunner` рассчитан на один туториал для локального игрока. Выделение общего исполнителя квестов (с туториалом как одним из видов квестов) - главный риск рефакторинга. В мультиплеере состояние контрактов своё у каждого игрока.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Прогресс сохраняется в JSON игрока (`Contracts`, необязательно). Доска засевается от seed мира и игрового дня, так что все игроки видят одну и ту же доску. Контракты на доставку забирают предметы на сервере через систему контейнеров. Тесты в стиле `DefaultTutorialContentTests` проверяют, что каждый шаблон сводится к корректным задачам и id.

E2 Торговые аванпосты и торговцы Spacer

Трудоёмкость L **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Нейтральные аванпосты (prefab'ы структур, защищённые админом или неразрушимые), где NPC Spacer покупают и продают по прайс-листам `trade.json`. Валюта - предмет (`credit_chip`), поэтому она работает с инвентарями, дропом и правилами смерти. Цены сдвигаются в зависимости от того, что продают игроки (простая кривая предложения на сервер). Админы могут править цены через dashboard.

ОПИРАЕТСЯ НА

NPC Spacer (`Storage`, `ContainerKey.ForObject`), структуры (X1), блоки `Unbreakable` и `EntityProtection`, синхронизация контейнеров с проверкой на сервере, модули dashboard, `DisassemblyBalanceAudit`.

ОПЫТ ИГРОКА

Избыточная добыча обретает смысл, игроки в кооперативе могут специализироваться, а редкие предметы становятся доступны.

РИСКИ

Эксплойты экономики (скрафтить дёшево, продать дорого). Автоматический тест-аудит должен гарантировать, что ни одна цена продажи не превышает самую дешёвую стоимость крафта, как `DefaultCrusherRecipesTests`.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Сделки - это запрос `INetMessage`, который сервер проверяет (запас, цена, дистанция 6 м), прежде чем отправить результат. Цены хранятся в папке мира (`economy.json`, необязательно). Работает в headless-режиме.

E3 Исследования через анализ

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Необязательное правило GameSet: продвинутые чертежи изначально закрыты. Игроки открывают их, анализируя образцы (новые руды, инопланетные детали с заброшенных кораблей) в анализаторе или разбирая найденные предметы (реверс-инжиниринг). Открытия принадлежат игроку или команде.

ОПИРАЕТСЯ НА

`AnalyzerScanner`, рецепты разборки, чертежи `crafting.json` (новое необязательное поле `Unlock`), категории крафта, `PlayerActivity`, глава туториала *Iron age*.

ОПЫТ ИГРОКА

Прогресс через открытия: исследование вознаграждает знанием, а не только предметами.

РИСКИ

Существующие миры, где игроки уже всё знают, должны продолжать работать. Сохранение без списка открытий означает, что открыто всё, а Default GameSet закрывает только новый контент.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Набор открытий хранится в JSON игрока (`Unlocks`, отсутствует = всё) и уже доходит до клиентов в JSON подключившегося игрока. Нужно проверить, валидируется ли крафт на сервере; если нет, для проверки открытий нужен серверный запрос крафта.

M Мультиплеер: кооператив и PvP

Серверная часть зрелая (claims, правила, dashboard, аккаунты). Не хватает социального слоя: команд, общего прогресса и честного PvP. (3 идеи: M1, M2, M3)

M1 Команды

Трудоёмкость M **Приоритет** P1 **Влияние** ●●●●○

СУТЬ ИДЕИ

Лёгкие фракции. Игроки создают команду или вступают в неё. Участники команды автоматически получают членство в claims друг друга, общий список waypoints команды, могут возрождаться в DNA-реконструкторах команды, видят друг друга на радаре с именами, имеют командный канал чата (`/c`) и права на ящики.

ОПИРАЕТСЯ НА

Участники `ClaimRegistry`, `WaypointService` / `TagManager` (сейчас свои у каждого клиента), `DnaReconstructorRegistry`, `ChatService`, `PlayerListNetwork`, `ProtectionRules.CanEdit`, страница Players в dashboard.

ОПЫТ ИГРОКА

Кооператив ощущается как командная игра, а участников claim больше не нужно добавлять по одному.

РИСКИ

Сложность прав доступа. Одиночная игра не должна быть затронута.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

`crews.json` в папке мира (сервер). `CrewStateMessage` отправляется при подключении и при каждом изменении. Waypoints команды хранятся на сервере (`crew_waypoints.json`) и рассылаются при изменении. `ProtectionRules` получает проверку команды (дешёвый поиск в множестве). Модуль dashboard плюс обновление руководства по серверу.

M2 Осады claims (PvP-серверы)

Трудоёмкость L **Приоритет** P3 **Влияние** ●●●○○

СУТЬ ИДЕИ

На PvP-серверах claims перестают быть абсолютными. Во время окна рейдов, заданного сервером (или когда оплаченное время маяка истекает), атакующие могут прорваться в захваченную базу, уничтожив генератор щита, питаемый от собственной сети базы. Пробивные заряды наносят урон блокам с учётом прочности. Защитники получают оповещения в чате и в dashboard.

ОПИРАЕТСЯ НА

Оплаченное время `ClaimBeaconBlock` и `ClaimSettings`, `ServerRules` (`ExplosionsRespectClaims`), `ProtectionRules.IsDamageProtected`, `ElectricNetworkManager`, взрывчатка, dashboard.

ОПЫТ ИГРОКА

Осмысленное PvP с подготовкой вместо гриферства, пока игроки офлайн.

РИСКИ

Токсичность и гриферство, поэтому механика по умолчанию выключена и ограничена окнами по расписанию.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Новые поля `ClaimSettings` со значениями по умолчанию (`SiegeEnabled = false`, окна). Состояние щита - состояние блока. Все проверки остаются в `ProtectionRules` (в одном месте).

M3 Таблицы лидеров и цели сервера

Трудоёмкость S **Приоритет** P2 **Влияние** ●●●○○

СУТЬ ИДЕИ

Таблицы лидеров на основе `PlayerStatistics` (дистанция, блоки, посещённые секторы, смерти по причинам) в меню паузы и в dashboard, плюс общесерверные цели («доставить 10 000 titanium ingot к вратам»), которые открывают косметические награды вроде текстур брони и цветов табличек.

ОПИРАЕТСЯ НА

`PlayerStatistics` / `PlayerStatsTracker`, модули dashboard, текстуры `ArmorRenderer`, цвета табличек, объявления в чате.

ОПЫТ ИГРОКА

Общие цели сообщества на публичных серверах при небольших затратах.

РИСКИ

Статистику сообщает владелец, поэтому её можно подделать. Ранжировать стоит только ту статистику, которую сервер видит сам.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Сервер агрегирует статистику из сохранений игроков и отправляет её по запросу. Модуль dashboard плюс страница в `app.js`.

G Цели эндгейма

После титана и кораблей долгосрочной цели нет. Это итоговые проекты, которым нужны все остальные системы. (2 идеи: G1, G2)

G1 Мегапроект Belt Gate

Трудоёмкость L **Приоритет** P2 **Влияние** ●●●●●

СУТЬ ИДЕИ

Долгосрочная стройка: огромные многоклеточные врата (16x16x4), собранные из модулей. Каждому модулю нужны материалы позднего этапа (обскурий, тулиевые линзы, урановые стержни, микросхемы), а вратам - большое стабильное энергоснабжение. Заряженные врата открывают прыжок в новый регион пояса: дальний пояс секторов с более сложными биомами и новыми рудами. В мультиплеере это общесерверное событие. Корабли прыгают вместе с пассажирами.

ОПИРАЕТСЯ НА

Многоклеточные блоки (`Size` до 16) и `MultiblockLayout`, `ElectricNetworkManager` (генераторы до 1600), система секторов (8192 сектора по каждой оси оставляют много места), `EntitySectorTracker`, `ShipRider`, `World.PreloadAroundPlayer` / `WorldLoadingScreen`.

ОПЫТ ИГРОКА

Видимая конечная цель, которая задействует все системы: добычу, логистику, энергию, корабли и оборону (пока врата заряжаются, происходят события).

РИСКИ

Стоимость контента нового региона и перемещение больших кораблей между секторами (стриминг).

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Прыжок - это `SetPose` в дальний сектор с предзагрузкой за экраном загрузки. Новый регион - просто дальний пояс биомов (X2), так что новый генератор не нужен. Заряд врат - состояние блока плюс сериализатор (`gate_data`).

G2 Улей-левиафан (босс)

Трудоёмкость L **Приоритет** P3 **Влияние** ●●●●○

СУТЬ ИДЕИ

Редкий гигантский астероид (радиус 200+), в ядре которого находятся королева паразитов и сеть ульев. Чтобы добраться до неё, нужны туннели, освещение, кислородные аванпосты и оборона. Победа над королевой даёт реликвии: уникальные открытия чертежей или компоненты врат.

ОПИРАЕТСЯ НА

T1 (ульи), состояния поведения мобов с увеличенным HP и призывом, пресеты астероидов с полостями (`cheese`, `InternalCavities`), таблицы лута, открытия E3.

ОПЫТ ИГРОКА

Экспедиция в духе подземелья и общая цель для команды.

РИСКИ

ИИ босса в воксельных туннелях; баланс для одного игрока и для группы.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Королева - новый `IMobBrain`, зарегистрированный в `MobBrainFactory`. Астероид - новый пресет с гарантированной полостью в ядре. Состояние босса (жив или мёртв) сохраняется в NBT астероида (`boss_defeated`, по умолчанию false).

EV Мировые события

События, запускаемые сервером, оживляют пояс и дают игрокам повод готовиться между сессиями. (4 идеи: EV1, EV2, EV3, EV4)

EV1 Режиссёр мировых событий

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Серверный режиссёр планирует события из GameSet `events.json` (веса, перезарядки, порог прогресса игрока, условия биома) и объявляет их (чат, всплывающее уведомление в HUD, звук). Это каркас для EV2-EV4 с админской командой `event start <id>` и страницей в dashboard.

ОПИРАЕТСЯ НА

`TickTaskManager`, `ServerRules` (правило `world_events` через `ServerRulesService`), `ChatService`, модули dashboard, `MobSpawner`.

ОПЫТ ИГРОКА

Пояс ощущается живым, и возникают истории.

РИСКИ

События, бьющие по пустым или AFK-базам, кажутся нечестными, поэтому они нацелены только на регионы с активными игроками.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

События - классы `IWorldEvent` (start, update, end), зарегистрированные в фабрике, как `TutorialTaskFactory`. Состояние хранится в папке мира (`world_events.json`, необязательно). Клиенты получают только `WorldEventMessage` (id, фаза, позиция).

EV2 Метеорные потоки

Трудоёмкость M **Приоритет** P2 **Влияние** ●●●●○

СУТЬ ИДЕИ

Рой небольших каменных сущностей на скорости пересекает регион. Они бьют по базам и кораблям (импульсы `ShipCollider` плюс урон блокам), их могут сбивать турели, и они оставляют после себя фрагменты редкой руды.

ОПИРАЕТСЯ НА

Динамический камень - просто сущность с `ShipBody`, `AsteroidVoxelDataGenerator` с малым радиусом, `ExplosionSystem` для ударов, эффекты попаданий снарядов.

ОПЫТ ИГРОКА

Игроки готовят оборону, и это зрелищно.

РИСКИ

Столкновения множества тел нагружают CPU, поэтому камней не более 20, каждый не больше 2 чанков, и они деспавнятся после пролёта. Урон по базам в claims регулируется правилом.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Камни симулируются на сервере и передаются через `ShipStateMessage`. Это временные сущности, и они не сохраняются, если только не застряли внутри базы.

EV3 Солнечные вспышки

Трудоёмкость S **Приоритет** P2 **Влияние** ●●●○○

ПЕРЕСЕЧЕНИЕ Требует S3.

СУТЬ ИДЕИ

Предупреждение за 2 минуты, затем 60-90 с вспышки: незащищённые игроки снаружи получают радиацию (S3), электроника на открытых кораблях мерцает (двигатели слабеют), а дальность радара падает. Укрытие - герметичная комната или экранирующие блоки между игроком и направлением вспышки.

ОПИРАЕТСЯ НА

`RadiationField` из S3, кислородные комнаты, `VignetteController` и цветокоррекция (оттенок экрана), `RadarSettings`, канал фонового звука.

ОПЫТ ИГРОКА

Общий момент «все внутрь», который проверяет дизайн базы.

РИСКИ

Неудачное время во время долгих перелётов, поэтому время предупреждения растёт с расстоянием до ближайшей герметичной комнаты.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Построено на EV1 и S3. Направление вспышки передаётся в `WorldEventMessage`. Клиент применяет эффекты к своему игроку.

EV4 Сигналы бедствия и кометы

Трудоёмкость M **Приоритет** P3 **Влияние** ●●●○○

ПЕРЕСЕЧЕНИЕ Требует X1.

СУТЬ ИДЕИ

Сигнал бедствия: на некотором расстоянии появляется заброшенный корабль (X1) с меткой на радаре и таймером; у выжившего Spacer внутри заканчивается воздух, если его не спасти аптечкой или кислородом. **Комета:** временная ледяная сущность с редким метаном и тулием, которая 20 минут пересекает сектор.

ОПИРАЕТСЯ НА

X1, метки радара, NPC Spacer, система подарков предметов (`ItemGiftNetwork`), контракты E1.

ОПЫТ ИГРОКА

Спонтанные короткие приключения.

РИСКИ

Давление таймера в одиночной игре; таймер останавливается, пока игра на паузе.

ТЕХНИЧЕСКИЕ ЗАМЕТКИ (производительность, сеть, формат сохранений)

Типы событий внутри режиссёра EV1. Комета - временная динамическая сущность, которая удаляется из сохранений по истечении срока.

Roadmap по приоритетам

Порядок следует трём правилам: (1) сначала дать игрокам причины исследовать и автоматизировать, потому что это умножает ценность уже существующих кораблей, машин и труб; (2) строить общий фундамент один раз (prefab'ы структур, исполнитель квестов, режиссёр событий, автопилот) и переиспользовать его; (3) добавлять давление и угрозы только тогда, когда у игроков есть инструменты для ответа (турели раньше рейдов, укрытия раньше солнечных вспышек).



ширина полосы = относительная трудоёмкость

Фаза 1: Причина покинуть первый астероид (ближайшие 2 версии)

Сначала дешёвые функции исследования и производства с большой отдачей. X3 и X2 работают с сегодняшним контентом. X1 - самый крупный пункт фазы, он открывает путь многим последующим идеям.

1.	X3 Импульс сканера-разведчика	S-M	P1
2.	X2 Ресурсные биомы	M	P1
3.	T1 Гнёзда паразитов и заражение	M	P1
4.	A1 Auto-miner	M	P1
5.	S1 Генераторы на топливе и аккумуляторные банки	M	P1
6.	SH3 Навигационный компьютер: автопилот и круиз	M	P1
7.	M1 Команды	M	P1
8.	X1 Заброшенные корабли и структуры	L	P1

Фаза 2: Экономика пояса и инженерия (следующие 2-3 версии)

E1 превращает движок туториала в бесконечную систему целей. Стыковка и топливо делают корабли частью экономики, а сигналы и турели углубляют базы.

1.	E1 Доска контрактов (на движке туториала)	M	P1
2.	SH1 Стыковочные коннекторы и посадочные захваты	L	P1
3.	SH2 Ракетное топливо и топливные баки	M	P2
4.	A2 Сигнальные каналы: датчики и исполнители	L	P2
5.	A3 Логистика: сортировщик, таблички запасов, терминал хранилищ	M	P2
6.	E3 Исследования через анализ	M	P2
7.	T2 Автоматические оборонительные турели	M	P2
8.	S4 Комфорт и настроение	S-M	P2
9.	M3 Таблицы лидеров и цели сервера	S	P2

Фаза 3: Живой и опасный пояс (среднесрочно)

Режиссёр событий плюс настоящие враги. T3 и SH4 - два пункта размера XL: их стоит планировать после того, как SH3, X1 и T2 станут стабильными.

1.	EV1 Режиссёр мировых событий	M	P2
2.	EV2 Метеорные потоки	M	P2
3.	S3 Радиация	M	P2
4.	EV3 Солнечные вспышки	S	P2
5.	S2 Пробоины корпуса и декомпрессия	M	P2
6.	T3 Дроны-мусорщики и пиратские корабли	XL	P2
7.	E2 Торговые аванпосты и торговцы Spacer	L	P2
8.	SH4 Модель повреждений кораблей и разделение	XL	P2

Фаза 4: Эндгейм (долгосрочно)

Итоговые цели, использующие все предыдущие системы, и необязательная глубина PvP для публичных серверов.

1.	G1 Мегапроект Belt Gate	L	P2
2.	G2 Улей-левиафан (босс)	L	P3
3.	M2 Осады claims (PvP-серверы)	L	P3
4.	A4 Грузовая катапульта (mass driver)	L	P3
5.	SH5 Тяговый луч и резак для разборки	M	P3
6.	EV4 Сигналы бедствия и кометы	M	P3

Общий фундамент и зависимости

Сделать сначала	Открывает или упрощает
X1 Заброшенные корабли	E2 Торговые аванпосты, T3 Пираты, EV4 Сигналы бедствия (общий конвейер prefab'ов)
X2 Ресурсные биомы	SH2 Топливо, SH3 Круиз, G1 Врата (направления и новый регион)
SH3 Автопилот	T3 ИИ пиратов (тот же контроллер полёта)
S3 Радиация	EV3 Солнечные вспышки
EV1 Режиссёр событий	EV2, EV3, EV4
T1 Гнёзда паразитов	G2 Улей-левиафан
M1 Команды	T2 «Свой-чужой» для турелей, M2 Осады, E3 открытия команды
E1 Контракты (исполнитель квестов)	E2 Торговые аванпосты, EV4 награды за спасение

Рекомендуемые три следующих шага

1. **X3 Импульс сканера-разведчика**: небольшой и только клиентский, он сразу делает добычу приятнее. Заодно проверяет новый шаг туториала со сканером.
2. **A1 Auto-miner + S1 Топливо и батареи**, вместе в одном релизе: первый настоящий производственный цикл (miner, трубы, печь, топливо обратно в реактор) - цель автора *automation*.
3. **X2 Ресурсные биомы**, затем **X1 Заброшенные корабли**: дать кораблям куда лететь. Когда `GenerationLoader.LoadStructures` заработает с prefab-сущностями, на нём позже построятся торговые аванпосты, пираты и сигналы бедствия.

Каждый шаг выходит вместе с: JSON-значениями по умолчанию в `Default` GameSet, серверным правилом, если механика добавляет давление, тестами в `Spacebox.Tests` для чистой логики (подсчёт очков, расчёт топлива, шаблоны контрактов, размещение prefab'ов), строкой в `DONE.txt`, а также обновлёнными `spec.md` и туториалом.